Data Structures and Algorithms — Lab

Topic

In this lab, we will revise the concepts of template Classes, Header files with template calss.

Objective

- Demonstrate the use of **C++ class templates**.
- Show how a single class can handle multiple data types without rewriting code.
- Organize programs using header and source files with templates.

Outcomes

- Understand how to use **templates with classes**.
- Learn why template implementations are usually written inside header files.
- Practice separating program structure into **header (.h) and source (.cpp) files**.
- Reinforce logic for finding maximum values in arrays.
- Demonstrate code reusability using templates.

Tasks

Students are required to complete the following tasks during lab time.

Task 1

Design a **template class Calculator** in C++ that can perform basic arithmetic operations for multiple numeric types. The class should:

1. Store two values of a generic type T.
2. Provide member functions to **add, subtract, multiply, and divide** the two values.
3. Include a function showAll() to display the results of all operations in a formatted way.
4. Support different numeric types (e.g., int, double) using C++ templates, so the same class can operate on integers, floating-point numbers, or other numeric types.
5. Demonstrate the functionality by creating objects of Calculator<int> and Calculator<double> in the main() function, and display the results.

Sample Input:	**Sample Output:**
Integer Calculator: 10, 2 Double Calculator: 7.5, 2.5	--- Integer Calculator --- Add:12 Subtract:8 Multiply:20 Divide: 5

	--- Double Calculator --- Add: 10 Subtract: 5 Multiply: 18.75 Divide: 3

Note the following constraints that must be handled:

- Division by zero should be handled appropriately (optional enhancement).
- Only numeric types are considered for template instantiation.

Task 2

In many real-life applications such as online games, academic results, and sports competitions, a **leaderboard** is used to rank participants based on their scores. These scores may be whole numbers (e.g., game points) or decimal values (e.g., GPA or ratings).

Design a **template class Leaderboard** in C++ that can manage and display rankings for different types of numeric scores.

Your program should:

1. Create a template class Leaderboard<T> that:
    - Stores up to 100 player names.
    - Stores corresponding scores of generic type T.
    - Keeps track of the number of players added.
2. Provide the following member functions:
    - addPlayer(string name, T score) → Adds a player's name and score to the leaderboard.
    - display() → Displays all players with their ranking position and scores.
    - getWinner() → Returns the name of the player with the highest score.
3. In the main() function:
    - Create one leaderboard for **integer scores** (e.g., game points).
    - Create another leaderboard for **double scores** (e.g., GPA).
    - Add at least three players in each leaderboard.
    - Display the leaderboard and print the winner/top performer.

Sample main for the integer LeaderBoard is created for you below:

```
int main() {
    // Integer scores
    Leaderboard<int> gameBoard;
    gameBoard.addPlayer("Alice", 1500);
    gameBoard.addPlayer("Bob",   2300);
```

```
gameBoard.addPlayer("Charlie", 1900);
gameBoard.display();
cout << "Winner: " << gameBoard.getWinner() << endl;
cout << endl;
 // Double scores (like GPA)
//complete the rest of the code yourself
…….}
```

Task 3

Your task is to design a **template-based Leaderboard system** in C++ by separating the class declaration and implementation using a **header file**.

Part 1: Header File Creation

1. Create a header file named Leaderboard.h.
2. Inside the header file:
    - Define a **template class Leaderboard<T>**.
    - Declare:
        - An array of names (maximum size 100).
        - An array of scores of type T.
        - An integer variable to track the number of players.
    - Define the following member functions inside the header file:
        - addPlayer(string name, T score)
        - display()
        - getWinner()

Note: Since this is a template class, all function definitions must remain inside the header file.

Part 2: Main File implementation

3. Create a separate file main.cpp.
4. Include the header file using : #include "Leaderboard.h”

In main do the following tasks:

- Create a `Leaderboard<int>` object for game scores.
- Add at least three players and display the leaderboard.

- Print the winner.
- Create a `Leaderboard<double>` object for GPA scores.
- Add at least three students and display the leaderboard.
- Print the top student.

Task 4: Stretch Your Thinking

If you create Calculator<string>, in Task1 what problem might occur and why?

Test , run and write which function causes problem , which functions will run ok.

---------- Great Job, Rest Time😊 ----------